CSCI-UA 60 Assignment 4: Normalization

Amina Salamatolla (as20312)

Exercise 1: Happy Supplies Parts Order Form

a. Assumptions

1. The form has no order number. Since surrogate keys are not allowed, an order is identified by the customer plus the date and time it was placed. A customer cannot place two separate orders at the same date and time.
2. customerNo (e.g. HG54587) is unique per customer and determines customerName and customerType. Two customers may share a name.
3. Each order is handled by exactly one employee. Only the employee's name appears on the form and there is no employee number, so employeeName is kept as a plain attribute of the order. Since employees help any customer, the employee depends on the order, not on the customer.
4. partNo is unique per part and determines partName, partType and unitPrice. The unit price is the company's standard list price for the part, not a price negotiated per order.
5. All stock of a given part is stored in one cage, so partNo $\rightarrow$ cageCode. A cage can hold many parts. In the sample, every Plumbing part happens to sit in cage G413, but that is a coincidence of this form; a cage is not restricted to one part type, so partType does not determine cageCode.
6. A part appears at most once per order; ordering more of it increases quantityOrdered instead of adding a second line.
7. Line totals and an order total are not on the form and are derivable (quantity $\times$ price), so they are not stored.

b/c. Normalization process

Attributes identified on the form

customerName, customerNo, customerType, orderDate, orderTime, employeeName (order header, appear once per form) and partNo, partName, partType, cageCode, quantityOrdered, unitPrice (one set per order line).

Step 0: Unnormalized Form (UNF)

All the form data in a single table. The six part attributes form a repeating group: one order holds four sets of part data, so the header values appear once and the row "contains" four part entries. This is not in any normal form.

PartsOrderForm(customerNo, customerName, customerType, orderDate, orderTime, employeeName, {partNo, partName, partType, cageCode, quantityOrdered, unitPrice})

Braces { } mark the repeating group.

customerNo	customerName	customerType	orderDate	orderTime	employeeName	partNo	partName	partType	cageCode	quantityOrdered	unitPrice
HG54587	Jeff Peterson	Consumer	7/1/2024	10:30am	D. Harrison	10654	Float Control	Plumbing	G413	4	12
						10456	Modulator	Electrical	H433	3	7
						10776	Hose Assembly	Plumbing	G413	7	9
						10657	Float Assembly	Plumbing	G413	5	10

Step 1: First Normal Form (1NF)

Change: the repeating group is removed by flattening. The order header values are copied onto every order line so that each cell holds a single atomic value and there is one row per part per order.

Primary key: one row is one part on one order. The order is identified by (customerNo, orderDate, orderTime) (assumption 1), and a part appears once per order (assumption 6), so the PK is (customerNo, orderDate, orderTime, partNo).

PartsOrderForm(**customerNo**, **orderDate**, **orderTime**, **partNo**, customerName, customerType, employeeName, partName, partType, cageCode, quantityOrdered, unitPrice)

customerNo	**orderDate**	**orderTime**	**partNo**	customerName	customerType	employeeName	partName	partType	cageCode	quantityOrdered	unitPrice
HG54587	7/1/2024	10:30am	10654	Jeff Peterson	Consumer	D. Harrison	Float Control	Plumbing	G413	4	12
HG54587	7/1/2024	10:30am	10456	Jeff Peterson	Consumer	D. Harrison	Modulator	Electrical	H433	3	7
HG54587	7/1/2024	10:30am	10776	Jeff Peterson	Consumer	D. Harrison	Hose Assembly	Plumbing	G413	7	9
HG54587	7/1/2024	10:30am	10657	Jeff Peterson	Consumer	D. Harrison	Float Assembly	Plumbing	G413	5	10

Functional dependencies in the 1NF table:

FD1 (full): customerNo, orderDate, orderTime, partNo → quantityOrdered
FD2 (partial): customerNo → customerName, customerType
FD3 (partial): customerNo, orderDate, orderTime → employeeName
FD4 (partial): partNo → partName, partType, cageCode, unitPrice

The table is in 1NF but not 2NF: FD2, FD3 and FD4 are partial dependencies (non-key attributes that depend on only part of the composite key). The consequence is visible in the data: "Jeff Peterson / Consumer / D. Harrison" is repeated four times, and if this is the only order containing part 10456, deleting the order would also delete the fact that part 10456 exists and lives in cage H433 (delete anomaly). A new part cannot be recorded until someone orders it (insert anomaly).

Step 2: Second Normal Form (2NF)

Change: each partial dependency is moved into its own relation, keyed by the part of the PK that its attributes depend on. The determinant stays behind in the original relation so the data can be joined back.

- FD2 → customerName, customerType move to a new relation keyed by customerNo.
- FD3 → employeeName moves to a new relation keyed by (customerNo, orderDate, orderTime), i.e. one row per order.
- FD4 → partName, partType, cageCode, unitPrice move to a new relation keyed by partNo.
- Only quantityOrdered depends on the full key, so it stays with the four key attributes.

Customer(**customerNo**, customerName, customerType)

customerNo	customerName	customerType
HG54587	Jeff Peterson	Consumer

CustomerOrder(**customerNo**, **orderDate**, **orderTime**, employeeName)

customerNo	**orderDate**	**orderTime**	employeeName
HG54587	7/1/2024	10:30am	D. Harrison

Part(**partNo**, partName, partType, cageCode, unitPrice)

partNo	partName	partType	cageCode	unitPrice
10654	Float Control	Plumbing	G413	12
10456	Modulator	Electrical	H433	7
10776	Hose Assembly	Plumbing	G413	9
10657	Float Assembly	Plumbing	G413	10

OrderLine(**customerNo**, **orderDate**, **orderTime**, **partNo**, quantityOrdered)

customerNo	**orderDate**	**orderTime**	**partNo**	quantityOrdered
HG54587	7/1/2024	10:30am	10654	4
HG54587	7/1/2024	10:30am	10456	3
HG54587	7/1/2024	10:30am	10776	7
HG54587	7/1/2024	10:30am	10657	5

Every non-key attribute now depends on the whole PK of its relation, so all four relations are in 2NF.

Step 3: Third Normal Form (3NF)

Check for transitive dependencies (a non-key attribute determining another non-key attribute):

- **Customer:** customerName does not determine customerType (names are not unique, assumption 2). No transitive dependency.
- **CustomerOrder:** only one non-key attribute, so none is possible.
- **Part:** partType → cageCode looks true in the sample, but by assumption 5 a cage is not tied to a part type, and no other non-key attribute determines another. No transitive dependency.
- **OrderLine:** only one non-key attribute.

Change: none needed. The 2NF relations are already in 3NF. In 3NF the foreign keys are marked:

Customer(**customerNo**, customerName, customerType)

Part(**partNo**, partName, partType, cageCode, unitPrice)

CustomerOrder(***customerNo***, **orderDate**, **orderTime**, employeeName)

OrderLine(***customerNo***, ***orderDate***, ***orderTime***, ***partNo***, quantityOrdered)

c/d. Final 3NF Relations and Keys

Relation	Primary key	Foreign key(s)
Customer	customerNo	—
Part	partNo	—
CustomerOrder	(customerNo, orderDate, orderTime)	customerNo → Customer(customerNo)
OrderLine	(customerNo, orderDate, orderTime, partNo)	(customerNo, orderDate, orderTime) → CustomerOrder partNo → Part(partNo)

The relation is named CustomerOrder instead of Order because ORDER is a reserved word in SQL.

Exercise 2: Panacea Mental Health Corporation

a. Assumptions

1. staffNo is unique per therapist and determines therapistName. patNo is unique per patient and determines patName. Names are not unique.
2. The composite column "appointment" is split into its two components, appDate and appTime. This is not a new field; it makes the existing value atomic.
3. A therapist works at only one branch on a given day (stated), so (staffNo, appDate) $\rightarrow$ branchNo. The same therapist may work at different branches on different days (S1032: M15 on 9/14, B13 on 9/15).
4. A therapist sees only one patient at a given date and time, so (staffNo, appDate, appTime) identifies an appointment.
5. A patient can only be in one appointment at a given date and time (they may have several appointments in a day, at different times), so (patNo, appDate, appTime) is also unique. This is a candidate key.
6. Each appointment is a single time slot at the start time shown; no end time or duration is recorded.
7. There is no branch data other than branchNo itself, so no separate Branch relation is created (that would only be a one-column list of codes).

b/c. Normalization process

Step 0: Starting Table (UNF)

All the data as given. The heading "appointment" groups two sub-values (date and time) under one composite attribute, so the table is not yet in 1NF as drawn.

TherapistAppointment(staffNo, therapistName, patNo, patName, appointment(date, time), branchNo)

				appointment		
staffNo	therapistName	patNo	patName	date	time	branchNo
S1011	Fred Smith	P100	Lily White	9/12/2022	10:00	M15
S1011	Fred Smith	P105	Jill Baker	9/12/2022	12:00	M15
S1024	Heidi Pierce	P108	Andy McKee	9/12/2022	10:00	Q10
S1024	Heidi Pierce	P108	Andy McKee	9/14/2022	14:00	Q10
S1032	Richard Levin	P105	Jill Baker	9/14/2022	16:30	M15
S1032	Richard Levin	P110	Jimmy Winter	9/15/2022	18:00	B13

Step 1: First Normal Form (1NF)

Change: the composite appointment is replaced by the atomic attributes appDate and appTime. There are no repeating groups (each row is already one appointment), so every cell is now single-valued.

Primary key: two candidate keys exist: (staffNo, appDate, appTime) and (patNo, appDate, appTime). I choose (staffNo, appDate, appTime) because staffNo is part of the determinant of branchNo, which keeps the decomposition clean. (patNo, appDate, appTime) remains an alternate key.

TherapistAppointment(**staffNo**, **appDate**, **appTime**, therapistName, patNo, patName, branchNo)

staffNo	**appDate**	**appTime**	therapistName	patNo	patName	branchNo
S1011	9/12/2022	10:00	Fred Smith	P100	Lily White	M15
S1011	9/12/2022	12:00	Fred Smith	P105	Jill Baker	M15
S1024	9/12/2022	10:00	Heidi Pierce	P108	Andy McKee	Q10
S1024	9/14/2022	14:00	Heidi Pierce	P108	Andy McKee	Q10
S1032	9/14/2022	16:30	Richard Levin	P105	Jill Baker	M15
S1032	9/15/2022	18:00	Richard Levin	P110	Jimmy Winter	B13

Functional dependencies in the 1NF table:

FD1 (full): staffNo, appDate, appTime → patNo, patName, therapistName, branchNo
FD2 (partial): staffNo → therapistName
FD3 (partial): staffNo, appDate → branchNo
FD4 (transitive): patNo → patName
FD5 (candidate key): patNo, appDate, appTime → staffNo

Not in 2NF: FD2 and FD3 are partial dependencies on the PK. The data shows the cost: "Fred Smith" and "Heidi Pierce" are stored twice each, and the fact that S1011 works at M15 on 9/12 is stored on every one of that day's appointments, so it could be updated in one row and not the other.

Step 2: Second Normal Form (2NF)

Change: the partially dependent attributes are moved out with copies of their determinants.

- FD2 → therapistName moves to a relation keyed by staffNo.
- FD3 → branchNo moves to a relation keyed by (staffNo, appDate): one row per therapist per working day.
- patNo and patName depend on the full key, so they stay in the appointment relation for now.

Therapist(**staffNo**, therapistName)

staffNo	therapistName
S1011	Fred Smith
S1024	Heidi Pierce
S1032	Richard Levin

TherapistSchedule(**staffNo**, **appDate**, branchNo)

staffNo	**appDate**	branchNo
S1011	9/12/2022	M15
S1024	9/12/2022	Q10
S1024	9/14/2022	Q10
S1032	9/14/2022	M15
S1032	9/15/2022	B13

Appointment(**staffNo**, **appDate**, **appTime**, patNo, patName)

staffNo	**appDate**	**appTime**	patNo	patName
S1011	9/12/2022	10:00	P100	Lily White
S1011	9/12/2022	12:00	P105	Jill Baker
S1024	9/12/2022	10:00	P108	Andy McKee
S1024	9/14/2022	14:00	P108	Andy McKee
S1032	9/14/2022	16:30	P105	Jill Baker
S1032	9/15/2022	18:00	P110	Jimmy Winter

All three are in 2NF. Note that TherapistSchedule has 5 rows while the original table had 6: S1011's two appointments on 9/12 collapse into one branch fact.

Step 3: Third Normal Form (3NF)

Check: Therapist and TherapistSchedule each have a single non-key attribute, so they cannot contain a transitive dependency. Appointment does: staffNo, appDate, appTime $\rightarrow$ patNo $\rightarrow$ patName (FD4). patName depends on a non-key attribute, so Jill Baker's name is stored on every appointment she has.

Change: patName moves to a new relation keyed by patNo; patNo stays in Appointment as a foreign key.

Therapist(**staffNo**, therapistName)

staffNo	therapistName
S1011	Fred Smith
S1024	Heidi Pierce
S1032	Richard Levin

Patient(**patNo**, patName)

patNo	patName
P100	Lily White
P105	Jill Baker
P108	Andy McKee
P110	Jimmy Winter

TherapistSchedule(***staffNo***, **appDate**, branchNo)

staffNo	**appDate**	branchNo
S1011	9/12/2022	M15
S1024	9/12/2022	Q10
S1024	9/14/2022	Q10
S1032	9/14/2022	M15
S1032	9/15/2022	B13

Appointment(***staffNo***, ***appDate***, **appTime**, *patNo*)

staffNo	***appDate***	**appTime**	*patNo*
S1011	9/12/2022	10:00	P100
S1011	9/12/2022	12:00	P105
S1024	9/12/2022	10:00	P108
S1024	9/14/2022	14:00	P108
S1032	9/14/2022	16:30	P105
S1032	9/15/2022	18:00	P110

In Appointment, (staffNo, appDate) is a foreign key to TherapistSchedule. This enforces the business rule that a therapist can only have an appointment on a day they are scheduled at a branch, and the branch of any appointment is found through that join. staffNo in TherapistSchedule is a foreign key to Therapist.

c/d. Final 3NF Relations and Keys

Relation	Primary key	Foreign key(s)
Therapist	staffNo	—
Patient	patNo	—
TherapistSchedule	(staffNo, appDate)	staffNo → Therapist(staffNo)
Appointment	(staffNo, appDate, appTime) alternate key: (patNo, appDate, appTime)	(staffNo, appDate) → TherapistSchedule(staffNo, appDate) patNo → Patient(patNo)

Exercise 3: Maid Better Temp Agency

a. Assumptions

1. eNo is unique per employee (stated) and determines eName. Names such as "Smith J" are not unique.
2. contractNo is unique per contract. Each contract applies to exactly one event (stated), so contractNo → eventNo. An event can have several contracts (H25 has C1024 and C1026).
3. eventNo is unique per event, and each event takes place at one location, so eventNo → eventLoc. Several events can share a location.
4. hours is the total number of hours one employee has worked on one contract, so (eNo, contractNo) → hours. There is one row per employee per contract; no date of work is recorded.
5. An employee can work on many contracts and a contract can have many employees (1135 works on C1024, C1025 and C1026).

b/c. Normalization process

Step 0: Starting Table

All the data as given, in one table:

EmployeeHours(eNo, contractNo, hours, eName, eventNo, eventLoc)

eNo	contractNo	hours	eName	eventNo	eventLoc
1135	C1024	16	Smith J	H25	Queens
1057	C1024	24	Hocine D	H25	Queens
1068	C1025	28	White T	H4	Yonkers
1135	C1025	15	Smith J	H4	Yonkers
1135	C1026	10	Smith J	H25	Queens

Step 1: First Normal Form (1NF)

Change: none to the structure. Every cell already holds one atomic value and there are no repeating groups, so the table is already in 1NF once a primary key is chosen.

Primary key: eNo alone is not unique (1135 appears 3 times) and neither is contractNo (C1024, C1025 twice). Each row records one employee's hours on one contract, so the PK is (eNo, contractNo).

EmployeeHours(**eNo**, **contractNo**, hours, eName, eventNo, eventLoc)

eNo	**contractNo**	hours	eName	eventNo	eventLoc
1135	C1024	16	Smith J	H25	Queens
1057	C1024	24	Hocine D	H25	Queens
1068	C1025	28	White T	H4	Yonkers
1135	C1025	15	Smith J	H4	Yonkers
1135	C1026	10	Smith J	H25	Queens

Functional dependencies in the 1NF table:

FD1 (full): eNo, contractNo → hours
FD2 (partial): eNo → eName
FD3 (partial): contractNo → eventNo, eventLoc
FD4 (transitive): eventNo → eventLoc

Not in 2NF: FD2 and FD3 are partial dependencies. "Smith J" is stored three times, and "H25 / Queens" is repeated for every employee on C1024.

Step 2: Second Normal Form (2NF)

Change:

- FD2 → eName moves to a relation keyed by eNo.
- FD3 → eventNo and eventLoc move to a relation keyed by contractNo.
- hours depends on the full key and stays.

Employee(**eNo**, eName)

eNo	eName
1135	Smith J
1057	Hocine D
1068	White T

Contract(**contractNo**, eventNo, eventLoc)

contractNo	eventNo	eventLoc
C1024	H25	Queens
C1025	H4	Yonkers
C1026	H25	Queens

EmployeeContract(**eNo**, **contractNo**, hours)

eNo	**contractNo**	hours
1135	C1024	16
1057	C1024	24
1068	C1025	28
1135	C1025	15
1135	C1026	10

All non-key attributes now depend on the whole PK, so all three are in 2NF.

Step 3: Third Normal Form (3NF)

Check: Employee and EmployeeContract each have one non-key attribute. Contract has a transitive dependency: contractNo → eventNo → eventLoc (FD4). "H25 / Queens" is stored once for C1024 and again for C1026, so changing the venue of H25 would need two updates, and if C1025 were deleted the location of event H4 would be lost.

Change: eventLoc moves to a new relation keyed by eventNo; eventNo stays in Contract as a foreign key.

Employee(**eNo**, eName)

eNo	eName
1135	Smith J
1057	Hocine D
1068	White T

Event(**eventNo**, eventLoc)

eventNo	eventLoc
H25	Queens
H4	Yonkers

Contract(**contractNo**, *eventNo*)

contractNo	*eventNo*
C1024	H25
C1025	H4
C1026	H25

EmployeeContract(***eNo***, ***contractNo***, hours)

eNo	***contractNo***	hours
1135	C1024	16
1057	C1024	24
1068	C1025	28
1135	C1025	15
1135	C1026	10

c/d. Final 3NF Relations and Keys

Relation	Primary key	Foreign key(s)
Employee	eNo	—
Event	eventNo	—
Contract	contractNo	eventNo → Event(eventNo)
EmployeeContract	(eNo, contractNo)	eNo → Employee(eNo) contractNo → Contract(contractNo)